理财知识图谱 + 对话式智能体站点

项目说明

2026-09-23 · 第一版

一、一句话，和它给谁用

一个**跑在自己电脑上**的理财入门站，做给**完全不懂金融**的人。打开就有东西可看：217 个理财词的拆解图谱、一个能用大白话问问题的多角色助手、几套能动手的模拟工具。

不注册、不登录、不填密钥也能用大部分功能 —— 投资画像、引导标记、模拟持仓全部住在浏览器自己的 `localStorage` 里，**服务端零用户状态**：没有账号、没有会话、没有 cookie，服务端连“谁是谁”这个概念都不存在。

二、打开能干什么

全站 **17 条页面路由 + 2 个静态辅页面**（图谱页、指南页）。按“你想干什么”分四组：

① 想知道这是个什么东西

地址	打开能做什么
`/`	主页，所有模块的卡片入口
`/graph.html`	**217 个理财词的交互图谱**：打开就 1:1 居中在「基金」上，点任意词展开它的解释 —— 一句话、例子、易混辨析、对比边各占一栏
`/guide.html`	使用指南，站内每个链接都指向真地址
`/onboarding`	第一次进来时的引导（“看过没有”这个标记住在你自己浏览器里）
`/legacy`	上一版页面，留着当**已知为真的参照物**：新页面哪里不对，跟它比

② 想查一个词、或者问一句

地址	打开能做什么
`/agent`	用大白话问一句理财问题，看它怎么答（答完必带风险提示）

地址	打开能做什么
/agents	**九个角色的议会**：问一句 → 九个角色各自表态 → 多空交锋 → 动态权重表 → 最终裁决。九个角色分两轮，第二轮要正面回应第一轮里立场相反的那位。风控官那条发言是**规则算出来的**（事件里标着 source: rule），不是模型说的 —— 模型不能给自己发免责声明
/psych	八个常见的投资偏差 + 「说说你现在的处境」；措辞一律是“你可能正在…”，**不下诊断**

③ 想动手

地址	打开能做什么
/profile	填你的兴趣与区间。页面上明写这**不是**风险承受能力
/assess	十道题 → C1–C5 等级。结果页会**把你的测出来的等级和自述那一项并排对照**，并讲清“这个等级是从哪几件事推出来的”
/portfolio	持仓与权重 / 两两相关矩阵 / 算出来的指标 / **压力情景**。“这只是把风险摆出来，不是建议”
/game	模拟买卖，看当前持仓、交易历史、投资摘要
/research	**行情特征研判**：先选标的和买入日、**让你自己先下判断**，再揭晓“那天买、到今天”的实际结果。一个把“看后视镜”这件事摆到明处的设计

④ 想往里看 —— 这台机器自己是怎么运转的

地址	打开能做什么
/dash	四个市场的行情。“这一页只是行情，不是建议”
/ledger	**预测台账**：模型的历史战绩。命中率**永远和“无脑喊涨”并排显示**，并写着“这些记录存在哪儿”
/intraday	分钟级策略：标的与周期 → 多周期共振 / 入场·止损·目标 / 量能与分布 / 时段。服务端没做出来的那几件事**原样照抄给你看，不翻译**
/linkage	关联检测：采集健康度 / 这一轮的快照 / **异动雷达** / **它跟谁有关联**
/collect	采集监视：采集器 / 最近一轮的每个源 / **健康时间线** / 快照索引

（另有 /next 是旧链接落点，和 / 发同一份东西。/luxury-dash、/invest-sim 两页是设计系统的样例页，只在站内点得到、直接贴地址会 404。）

三、规模

层	是什么	规模（可核）
页面	一个壳 + 客户端路由 + 一个旧页面参照	**17 条**服务端页面路由（**16 条**发同一份 SPA 壳 + /legacy）

层	是什么	规模（可核）
后端	Flask	**903 行、21 条**接口
知识图谱	词条 + 分类，两层结构	**217 个词条**、289 个节点、625 条边、6 个族 14 个类
预测引擎	本地小模型 Kronos-small	24.7M 参数；实测在本机 RTX 4060 上预测 30 根 K 线用 204 ms
采集层	行情 / 新闻 / 榜单多源	4 个市场 × 5 类数据源
验证层	自动化判据 + 故障注入，分五套	整站 **69 条**；四个模块自检另有 **131 / 57 / 20 / 15 条**

四、三个能被你自己验证的能力

- **多角色议会，不是一个模型自说自话。** 一次提问开一场会：**9 个角色、两轮**，各自独立给方向和信心值，第二轮要正面回应第一轮里立场相反的那一位。**风控官那条发言是规则算出来的，不是模型说的**（事件里的 `source` 字段标着 `rule`）—— 模型不能给自己发免责声明，这是设计上的硬分工。
- **采集层四层降级，取不到就说取不到。** 从“随主体一起装好的 akshare”一路兜到浏览器渲染和社媒抓取，上层永远只看到一个结构化的失败描述，**绝不抛异常、也绝不因为真源取不到就换一份假数据出来**。实测四层全开：串行 13 秒，并发 4 路降到 3.5 秒。另有一条纪律值得单说：**桩开关只认服务端环境变量、不接受任何客户端参数** —— 一个客户端能打开的桩，就是一台伪造新闻的机器。
- **图谱里每个词都拆成了结构化字段**，不是一段散文：一句话、例子、易混辨析、对比边各占一栏。所以它现在能当词典查，将来也能直接喂给对话层当依据。

五、这个项目最想说的一件事：一次自带对照组的诚实评测

模型能出预测是这个方向的入场券，**而不是卖点**。真正值得看的是：我们把预测拿去做了一次**有对照组的**评测，然后如实公布了结论。

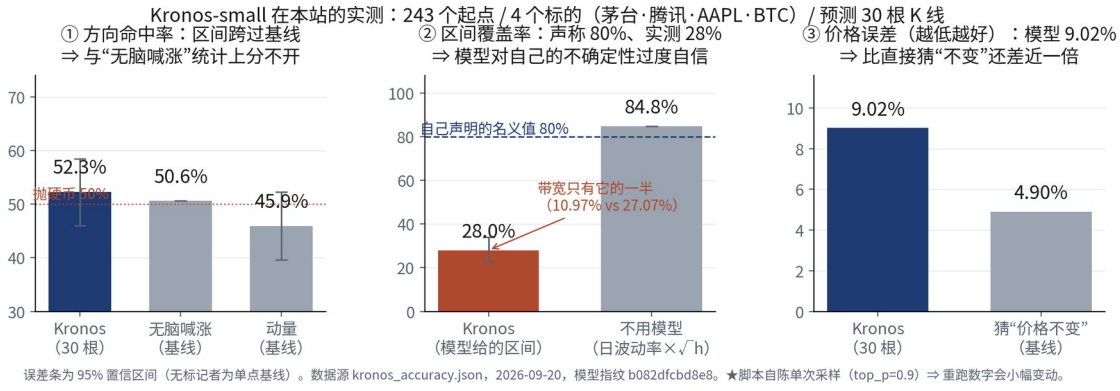


图 1　Kronos-small 在本站的实测：243 个起点、4 个标的（茅台·腾讯·AAPL·BTC）、预测未来 30 根 K 线。三个面板分别回答“方向准不准”“区间可不可信”“误差有多大”，每个都配了不用模型的基线作对照。

四条结论，都不好听：

- **方向命中率 52.3%，看起来赢过“无脑喊涨”的 50.6% —— 但 95% 置信区间是 [46.0, 58.5]，把基线包在里面。换句话说：这个差距和“抛硬币”统计上分不开。** 而“追动量”的基线是 45.9%，反而更差。
- **模型给的区间声称覆盖 80%，实测只覆盖 28.0%**，而且它的带宽只有对照带子的一半（10.97% 对 27.07%）。**模型对自己的不确定性过于自信** —— 而那个完全不用模型、只用日波动率乘出来的带子，实测覆盖 84.8%，校准得**好得多**。
- **价格误差 9.02%，而“直接猜价格不变”是 4.90%** —— 模型比不动还差近一倍。
- **而且这 52.3% 是“两个方向相反的结果”平均出来的**：在茅台上 60.0%（基线 43.1%），在苹果上 46.2% —— **而苹果那一格，两个基线分别是 56.9% 和 52.4%，模型比它们都差**。一个真有技能的东西不会长成这样。

所以这个项目的结论是：没有超额能力。 我们没有把这句话藏起来，而是把它印在产品的台账页上（命中率永远和“无脑喊涨”并排显示）。

我自己的判断是：**把这个结论如实写出来，比宣称“高精度”更有说服力** —— 因为它证明了评测这件事是被认真做过的，而不是挑了对自己有利的数字。评测脚本自己也声明了一条：单次采样（top_p=0.9），**重跑数字会小幅变动**，不假装可复现。

六、工程上的做法：每一条判据都必须能被“打红”

这个站的自动化判据分五套：**整站报告 69 条**，加上四个模块各自的自检 —— **关联检测 131 条、分钟级策略 57 条、采集器 20 条**（外加 3 条它自己标注“没有注入、没人替它们把关”的守卫）、**议会 15 条**。

这五个数不能相加：它们量的不是同一个层次 —— 一套查整站的路由与落盘，另一套查某个模块内部的算法行为，相加只会得到一个没有对应物的总数。

真正少见的不是条数，而是那条纪律：

> **每一条判据都必须配一个能把它打红的故障注入；配不出来的，要在代码里写明豁免声明和理由。**

否则判据就是装饰 —— 它绿得和被测代码无关。这条纪律本身也有判据守着（哪个文件的哪条判据没有注入、豁免声明是否过期，都会被查）。

配套的还有两条：**注释里不许出现和常量打架的说法**；**任何对外承诺都要有判据，不能只是一句自我声明**。举个例子：上面“服务端零用户状态”那句承诺，除了文档里写清楚，还有一条判据逐条检查落盘记录的键必须落在白名单里，并配了一条“写入点偷偷多记一个字段”的注入 —— 模拟将来有人顺手多写一句用户提问，而那种改动不会让任何别的判据变红。

七、明确做不到的（这一节是故意留的）

- **模型没有超额能力**，见第五节。它在本站的角色是“一个可以被观察的对象”，不是“一个能赚钱的工具”。

- **有几处数字是本站的教学口径，不是监管原文**：风险测评的题库、每题分值、以及 C1–C5 的分档线都是我们自己定的，页面上明写了这一句 —— “真实券商用的题目、权重、分档线各不相同，以你开户的那家为准”。一套看起来像官方问卷的东西如果不说这句，就是全站唯一一处没有出处的数字。
- **不构成投资建议，也不构成心理诊断**。 模型收尾那句里“不构成投资建议”六个字是**要求原样出现**的；心理那一页写明“这不是心理诊断，也不是投资建议”。
- **有一个已知缺口没修**：行情总览页在站内**点不进去**（只能手输地址），因为导航那一处漏了。这不是设计，是漏了，我们把它写在了文档里而不是藏起来。

八、想自己跑一遍

```
cd /home/ylf/licai-agent
.venv/bin/python app.py          # 起服务，默认 8848 端口
```

浏览器打开 `http://127.0.0.1:8848/`。不填密钥也能看图谱、查词、跑离线预测；想让它开口说话，再填一把自己的模型密钥（只存在本机 `.env`，页面上只显示掩码）。

验证层可以单独跑：`.venv/bin/python verify_agent.py --selftest` —— 全套判据加故障注入。**注意它很慢**（按记录整套跑完约两小时），因为它要把整站 69 条注入逐条真的打进代码里再验一遍。四个模块的自检可以各自单跑（`agent/linkage/_probe.py` 等），几秒到几分钟 —— 想当场看“判据确实能被打红”，跑模块自检就够了。整站那套不适合当场演示。

> 说明：本文里凡出现数字，都取自仓库内的实测产物（`kronos_accuracy.json`、`kronos_perf.json`、`graph_data.json`）、代码中的常量、或判据自己报出的读数，未使用估算值。判据条数是自己跑出来的（69 取自整站报告的“全部 69 条都够得着”，其余四套取自各自自检的收尾统计）。